

Node.js

1. How do you start a new Node.js project?

Run `npm init -y`.

This creates a default **package.json** file which stores project dependencies and metadata.

2. How do you read environment variables in Node.js?

Use `process.env.VARIABLE_NAME`.

To load them from a `.env` file, install **dotenv** and call `require('dotenv').config()`.

3. How do you create a simple HTTP server without Express?

```
const http = require('http');
http.createServer((req, res) => {
  res.end('Hello Node');
}).listen(3000);
```

This uses the built-in **http** module to create a basic web server.

4. How do you define a route `/ping` in Express?

```
app.get('/ping', (req, res) => res.send('pong'));
```

Used to verify server status or health checks.

5. How do you parse JSON request body in Express 4.16+?

Use middleware:

```
app.use(express.json());
```

This automatically parses incoming JSON data from requests.

6. What is middleware in Express?

Middleware are functions that execute between the request and response cycle.

They are used for **logging**, **authentication**, **error handling**, and **validation**.

7. How do you serve static files from the public directory?

```
app.use(express.static('public'));
```

This serves HTML, CSS, JS, and image files directly.

8. What is the difference between req.params and req.query?

- req.params stores **URL parameters** (/user/:id)
- req.query stores **query string parameters** (?page=2)

9. How do you handle 404 routes in Express?

```
app.use((req, res) => res.status(404).send('Not Found'));
```

This must be placed at the bottom of all routes.

10. How do you handle async errors in Express?

Use an error-handling middleware:

```
app.use((err, req, res, next) => {  
  res.status(500).send(err.message);  
});
```

11. How do you connect Node.js with MongoDB?

Using **Mongoose**:

```
mongoose.connect(process.env.MONGO_URI);
```

This creates a connection between your app and MongoDB database.

12. How do you enforce schema on MongoDB?

By defining a **Mongoose Model**, which ensures data follows a structure before saving.

13. What does \$set do in MongoDB updates?

It updates only specific fields of a document instead of replacing the entire document.

14. How do you perform “update if exists, else insert” in MongoDB?

Use `updateOne(query, update, { upsert: true })`.

15. What is the cluster module in Node.js?

It allows Node.js to use **multiple CPU cores** by creating worker processes — improving performance.

16. What does `process.nextTick()` do?

It defers a function’s execution until the **next iteration** of the event loop.

17. How do you read file contents asynchronously?

```
const fs = require('fs');
fs.readFile('test.txt', 'utf8', (err, data) => console.log(data));
```

18. How do you handle file uploads in Node.js?

Use **multer** middleware for handling multipart/form-data (e.g., image uploads).

19. How do you create a REST API route `/api/users` (GET)?

```
app.get('/api/users', async (req, res) => { ... });
```

20. How do you implement JWT in Node.js?

```
const token = jwt.sign({ id: user.id }, process.env.JWT_SECRET);
```

JWT is used for **authentication** and **authorization**.

21. How do you secure an Express app?

Use:

- **helmet()** for setting HTTP headers
- **cors()** to handle cross-origin requests
- **input validation** to prevent injection attacks

22. How do you enable CORS in Node.js?

```
const cors = require('cors');
```

```
app.use(cors());
```

23. What is the difference between require and import?

- require is used in **CommonJS** modules (Node.js default)
- import is used in **ES Modules**

24. How do you use environment variables in Node.js?

Add values in .env and read using:

```
require('dotenv').config();
console.log(process.env.DB_URL);
```

25. What is event-driven architecture in Node.js?

Node.js uses **EventEmitter** for handling asynchronous events using callbacks.

26. What is the difference between synchronous and asynchronous code?

Synchronous code blocks execution;

Asynchronous code allows non-blocking operations using callbacks, promises, or async/await.

27. What are streams in Node.js?

Streams handle continuous data flow efficiently — used for **reading/writing files** or **network communication**.

28. How do you export and import modules?

```
// export
module.exports = myFunction;
// import
const myFunction = require('./file');
```

29. What is the use of nodemon?

It automatically restarts the server when file changes are detected during development.

30. What are HTTP methods commonly used in REST API?

GET, POST, PUT, PATCH, DELETE — for CRUD operations.

31. How do you handle database errors in Node.js?

Use try...catch with async/await or .catch() with Promises.

32. What is package.json used for?

Stores metadata like project name, version, dependencies, and scripts.

33. What is package-lock.json?

It locks the exact versions of installed packages to ensure consistent installs across environments.

34. How do you improve Node.js performance?

Use caching (Redis), clustering, compression middleware, and async/non-blocking code.

35. How do you deploy a Node.js app?

Deploy using platforms like **Render, Vercel, AWS EC2, or Heroku**.
Set environment variables and use npm start as the entry command.

React.js

1. How do you create a new React project with Vite?

```
npm create vite@latest myapp -- --template react
```

This sets up a fast React environment using Vite as the build tool.

2. How do you initialize a state variable in React?

```
const [count, setCount] = useState(0);
```

3. What does useEffect do?

It allows you to perform side effects such as fetching data or DOM manipulation after rendering.

4. How to run useEffect only once on component mount?

```
useEffect(() => { fetchData(); }, []);
```

5. How to share state between two sibling components?

Lift the state up to their **common parent** or use **Context API**.

6. How do you conditionally render a component?

```
return loading ? <Spinner/> : <DataList/>;
```

7. How to pass data from parent to child?

```
<Child data={value} />
```

8. How to pass data from child to parent?

```
<Child onChange={(value) => setData(value)} />
```

9. How do you fetch data when the component mounts?

```
useEffect(() => {  
  fetch('/api/data')  
    .then(res => res.json())  
    .then(setData);  
}, []);
```

10. How to handle form inputs in React?

```
const [name, setName] = useState("");  
<input value={name} onChange={e => setName(e.target.value)} />
```

11. What is the key prop in React lists?

A unique key helps React efficiently update or re-render list items.

12. How do you use Context API for global state?

```
const AppContext = createContext();  
<AppContext.Provider value={user}>  
  <Child />  
</AppContext.Provider>
```

13. How to navigate programmatically using React Router?

```
const navigate = useNavigate();  
navigate('/home');
```

14. How do you handle route parameters like /users/:id?

```
const { id } = useParams();
```

15. How do you optimize re-renders in React?

Use `React.memo()`, `useCallback()`, or `useMemo()` to memoize components and values.

16. What is the difference between controlled and uncontrolled components?

Controlled inputs are managed by React state; uncontrolled ones use **refs**.

17. What is JSX?

A syntax that combines JavaScript and HTML-like code, compiled into `React.createElement()`.

18. How do you pass environment variables to React (Vite)?

Prefix them with **VITE_** and access them via `import.meta.env`.

19. How do you handle errors in React?

Use **Error Boundaries** (class components) or `try/catch` blocks in async functions.

20. What happens when you call `setState` or `setCount`?

React schedules a re-render with the updated state value.

21. Difference between `useEffect` and `useLayoutEffect`?

`useEffect` runs **after paint**, `useLayoutEffect` runs **before paint** (useful for DOM measurements).

22. How do you memoize a value in React?

```
const memoized = useMemo(() => expensiveCalc(data), [data]);
```

23. What is prop drilling and how do you prevent it?

Prop drilling is passing props deeply through components. Prevent it using **Context API** or **state management tools** like Redux or Zustand.

24. How do you show a component only if the user is logged in?

```
return isLoggedIn ? <Dashboard/> : <Login/>;
```

25. How do you use fragments in React?

```
<>
  <p>Line 1</p>
  <p>Line 2</p>
</>
```

Used to group multiple elements without adding an extra DOM node.

26. What is Virtual DOM?

A lightweight copy of the real DOM that React updates efficiently before applying changes to the actual DOM.

27. What are React Hooks?

Functions that let you use state and lifecycle features in functional components (e.g., `useState`, `useEffect`, `useContext`).

28. What is the difference between state and props?

- **State** is local to a component.
- **Props** are passed from parent to child and are read-only.

29. What is lazy loading in React?

Used to load components on demand using `React.lazy()` and `Suspense` to optimize performance.

30. What is the purpose of `useRef()`?

It allows direct access to DOM elements or stores mutable values that don't trigger re-renders.

31. How do you deploy React with a Node.js backend?

Build the frontend (`npm run build`) and serve it via **Express static middleware**, or deploy separately using **Vercel**, **Netlify**, or **Render**.

32. What is reconciliation in React?

The process by which React updates the DOM when state or props change using the **diffing algorithm**.

33. What are React keys and why are they important?

Keys help React identify elements in a list to re-render efficiently and prevent UI bugs.

34. What are synthetic events in React?

React wraps browser events into a cross-browser **SyntheticEvent** for consistency.

35. What is hydration in React?

It is the process of attaching React event handlers to existing server-rendered HTML (used in SSR apps).